

Roteiro do vídeo - parte do Paulo Victor

Duração: cerca de 4 minutos. A Isabela apresenta os slides; esta parte começa quando ela termina. Mostra o código, roda o programa explicando o resultado e testa o hash em um site gerador de bcrypt. Os valores de sal e hash mudam a cada execução; leia o que aparecer na tela.

Antes de gravar: (1) terminal na pasta do projeto; (2) `bcrypt_seminario.py` aberto em um editor; (3) o site gerador de bcrypt aberto em uma aba; (4) rodar uma vez para testar (leva uns 8 s até o menu).

1. Código (~0:50)

NA TELA: editor com `bcrypt_seminario.py`; rolar devagar pelos blocos

Agora vou mostrar o código. É um único arquivo, sem nenhum import. Ele tem cinco blocos. A: calcula os dígitos de pi com a fórmula de Machin. B: implementa a cifra Blowfish, com a função F, as 16 rodadas e a função `expandir_chave`, que é a operação central. C: o base64 próprio do bcrypt. D: o sal aleatório, com um gerador simples, só para demonstração. E: junta tudo em `hash_bruto`, `gerar_hash` e `verificar_senha`.

NA TELA: parar em `hash_bruto`, no laço `for _ in range(2 ** custo)`

Aqui está a lentidão proposital: o laço repete 2 elevado ao custo vezes duas chamadas de `expandir_chave`. Cada +1 de custo dobra o tempo.

2. Rodar e explicar o resultado (~1:30)

NA TELA: terminal: `python bcrypt_seminario.py`

Vou rodar o programa. A Parte 1 mostra o bcrypt passo a passo com a senha 'abc' e custo 4, que são 16 repetições. O sal são 16 bytes aleatórios e muda a cada execução. No passo 1 a senha vira 4 bytes: 'abc' mais um byte nulo. No passo 2 o estado começa com os dígitos de pi, `P[0]` igual a 243F6A88. No passo 3 misturamos sal e senha e o `P[0]` muda. No passo 4 repetimos as 16 voltas e o `P[0]` muda de novo; esse laço é a parte lenta. No passo 5 cifra-se 64 vezes um texto fixo, e no passo 6 montamos o hash final. Embaixo o programa mostra 17.385 cifragens, igual à fórmula, e a anatomia do hash: versão, custo 04, sal de 22 caracteres e hash de 31.

NA TELA: PARTE 2: hash 1 e hash 2, sal 1 e sal 2, True/True

A Parte 2 mostra o sal. A mesma senha, 'senha123', gerou dois hashes diferentes, porque cada um recebeu um sal aleatório. Os dois continuam válidos: a verificação dá True nos dois, porque o login lê o sal de dentro de cada hash.

NA TELA: PARTE 3: senhas A, B e C, hashes, ACEITO!

A Parte 3 mostra uma limitação: o bcrypt só usa os 72 primeiros bytes da senha. As senhas A e B só diferem depois do byte 72, e os hashes saem iguais. Já a C difere no primeiro byte, e o hash muda. O resultado é que a senha B consegue entrar na conta da senha A. Por isso, em sistemas reais, senhas longas passam por um pré-hash.

3. Força bruta em PIN (~0:40)

NA TELA: menu: digitar 3; PIN 07 (ou Enter); mostrar a tabela de projeção

No menu, a opção 3 simula um atacante que testa PINs de 2 dígitos, de 00 em diante, até acertar. O programa conta quantas cifragens ele gastou, cerca de 17 mil por tentativa.

Embaixo está a projeção: um PIN de 4 dígitos com custo 4 já dá centenas de milhões de cifragens, e com custo 12 são cerca de 245 vezes mais. O esforço cresce com o tamanho da senha e com o custo, e é isso que o bcrypt oferece.

4. Teste no site (~1:00)

NA TELA: menu: digitar 1, senha videoteste, custo 4; copiar o hash \$2b\$04\$...

Agora a prova externa. Vou gerar com o nosso programa o hash da senha 'videoteste' e copiá-lo.

NA TELA: no site gerador de bcrypt: aba de verificação (Check/Compare); colar senha e hash

Neste site gerador de bcrypt, colo a senha e o hash. O site informa que bate. Ou seja, um bcrypt feito por terceiros reconhece o hash do nosso código como válido.

NA TELA: no site: gerar hash de 'videoteste' com custo 4; no programa, menu 2: senha e hash colado

Agora o caminho inverso: gero um hash no site e o nosso programa o verifica na opção 2. Aparece 'Senha CORRETA'. Se eu mudar uma letra da senha, aparece 'INCORRETA'.

Resumindo: a nossa implementação em Python puro é compatível com o bcrypt oficial. Obrigado por assistir!

Dicas de gravação

- Rolar o terminal para cima ajuda a mostrar cada parte enquanto fala.
- Use custo 4 ou 5 no menu e no site. Custos altos demoram muito em Python puro.
- Use só senhas de teste no site, nunca senhas reais.
- Se o site gerar hash \$2a\$ ou \$2y\$, o programa aceita na verificação.
- Entreguem o vídeo no Classroom junto com o código, até as 19h do dia da apresentação.